

Digital System Design

Dr. Nazanin Ghasemian

Fall 2026 · Lecture 2



Combinational Logic- Intro

- Outputs depend only on current inputs (no memory)
- Decision-making (the “brain” that checks conditions)
 - Vending Machines
 - Digital Thermostat
 - Car Airbag Deployment System
 - Elevator Control

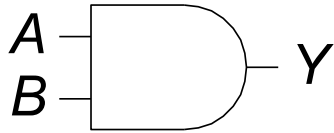
Combinational Logic

Basic building blocks:

- Logic gates:
 - Three 'primary' gates: AND, OR, NOT
 - Four 'derived' gates: XOR, NAND, NOR, XNOR

Combinational Logic-Primary Gates

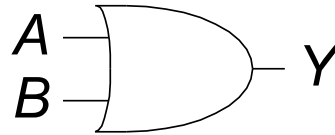
AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

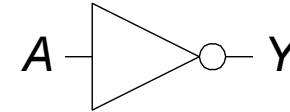
OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT

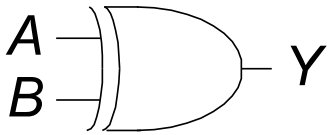


$$Y = \overline{A}$$

A	Y
0	1
1	0

Combinational Logic-Derived Gates

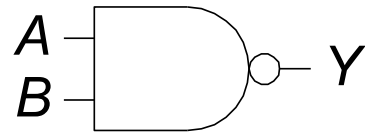
XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

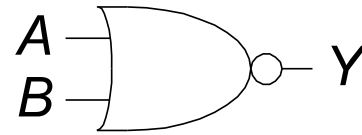
NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

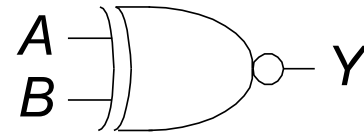
NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR



$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Combinational Logic- Common circuits

- Half Adder, Full Adder, Subtractor
- Encoders/Decoders
- Multiplexers (MUX)
- Comparator
- ...

Steps to Design a Combinational Circuit

Step 1: Define the Problem

- Clearly state the inputs and outputs.
- Understand what the circuit is supposed to do
- Example: A circuit that takes a 2-bit number and outputs 1 if it's prime.

Steps to Design a Combinational Circuit

Step 2: Identify Inputs and Outputs

- Decide how many input variables are needed and how many output signals.
- Example: A 2-bit prime detector $\rightarrow$ inputs = A, B; output = P (prime).

Steps to Design a Combinational Circuit

Step 3: Construct the Truth Table

- List all possible combinations of inputs.
- Write the desired output for each case.
- The truth table is the blueprint of your circuit.

Steps to Design a Combinational Circuit

Step 4: Derive Boolean Expressions

- From the truth table, write the output as a sum of minterms (SOP) or product of maxterms (POS).
- This gives a direct Boolean formula for the circuit.

Steps to Design a Combinational Circuit

Step 5: Simplify the Boolean Expression

- Use Boolean algebra or Karnaugh Maps (K-maps) to reduce the number of gates.
- Goal: Make the circuit smaller, faster, and cheaper.

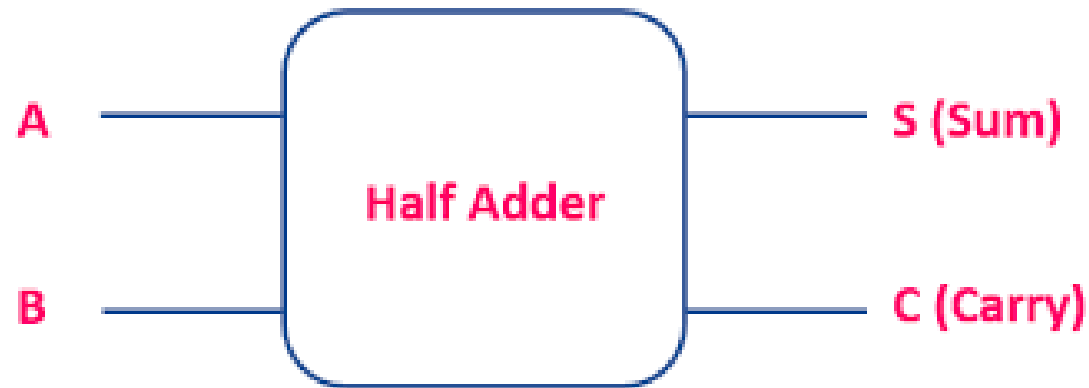
Step 6: Draw the Logic Diagram

- Convert the simplified Boolean expression into a gate-level circuit.

Half Adder

Half Adder (HA) is a combinational logic circuit.

- It adds two single-bit binary inputs (A and B).
- It produces two outputs: sum, carry



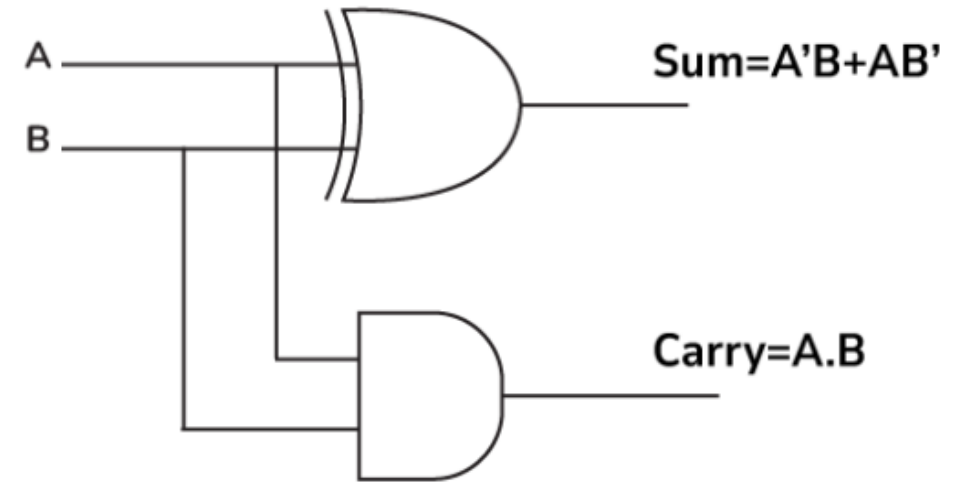
Half Adder

Truth Table

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = \bar{A}B + A\bar{B} = A \oplus B$$

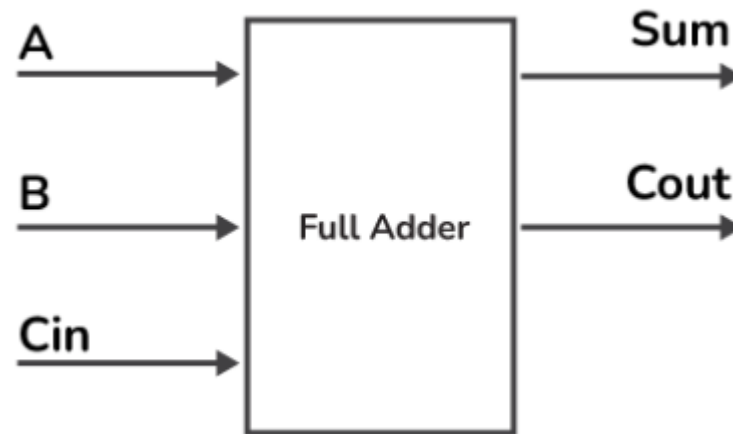
$$C = AB$$



Full Adder

A Full Adder (FA) is a combinational logic circuit.

- It adds three single-bit inputs: A, B, and Cin (carry-in).
- It produces two outputs: sum, carry



Full Adder

Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{aligned} S &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \\ &= \bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in}) \\ &= \bar{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \\ &= A \oplus B \oplus C_{in} \end{aligned}$$

$$\begin{aligned} C &= \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \\ &= BC_{in} + AC_{in} + AB \end{aligned}$$

Full Adder

Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

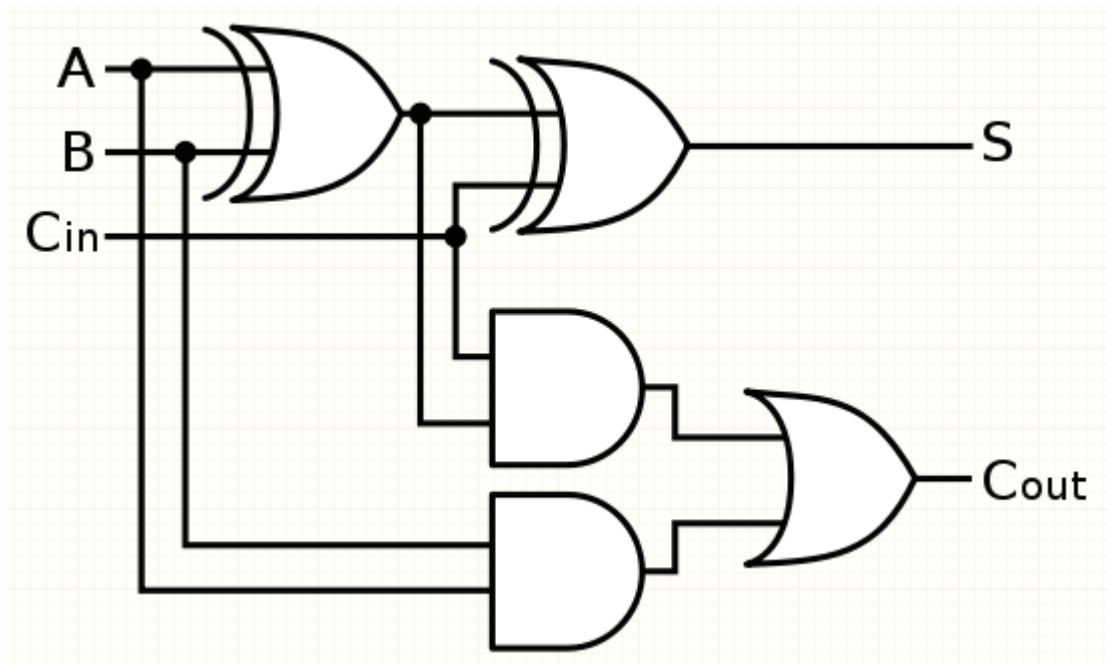
A \ BC _{in}	00	01	11	10
0	0 0	1 1	0 3	1 2
1	1 4	0 5	1 7	0 6

$$S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$$

A \ BC _{in}	00	01	11	10
0	0 0	0 1	1 3	0 2
1	0 4	1 5	1 7	1 6

$$C_{out} = AB + AC_{in} + BC_{in}$$

Full Adder



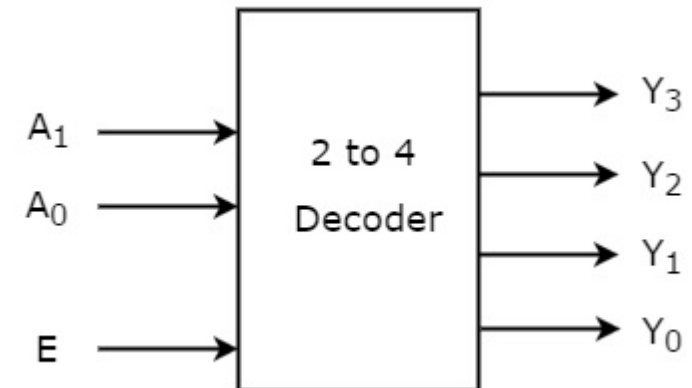
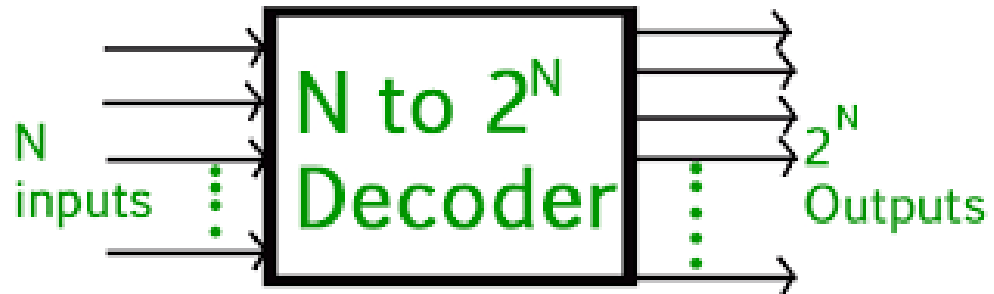
$$\begin{aligned} S &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \\ &= \bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in}) \\ &= \bar{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \\ &= A \oplus B \oplus C_{in} \end{aligned}$$

$$\begin{aligned} C &= \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in} \\ &= BC_{in} + AC_{in} + AB \end{aligned}$$

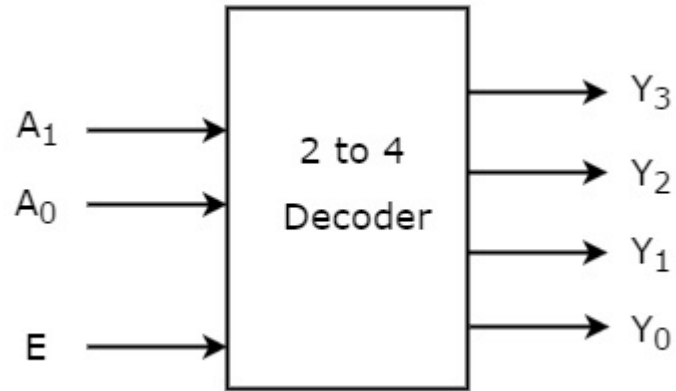
Decoder

A Decoder converts binary input codes into a unique one-hot output (only one output line is active at a time).

- General form: n -to- 2^n decoder (n input lines, 2^n output lines).
- Example: A 2-to-4 decoder takes 2 input bits and activates exactly one of its 4 outputs.



Decoder



Enable	INPUTS		OUTPUTS			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

$$Y_0 = E \overline{A_1} \overline{A_0}$$

$$Y_1 = E \overline{A_1} A_0$$

$$Y_2 = E A_1 \overline{A_0}$$

$$Y_3 = E A_1 A_0$$

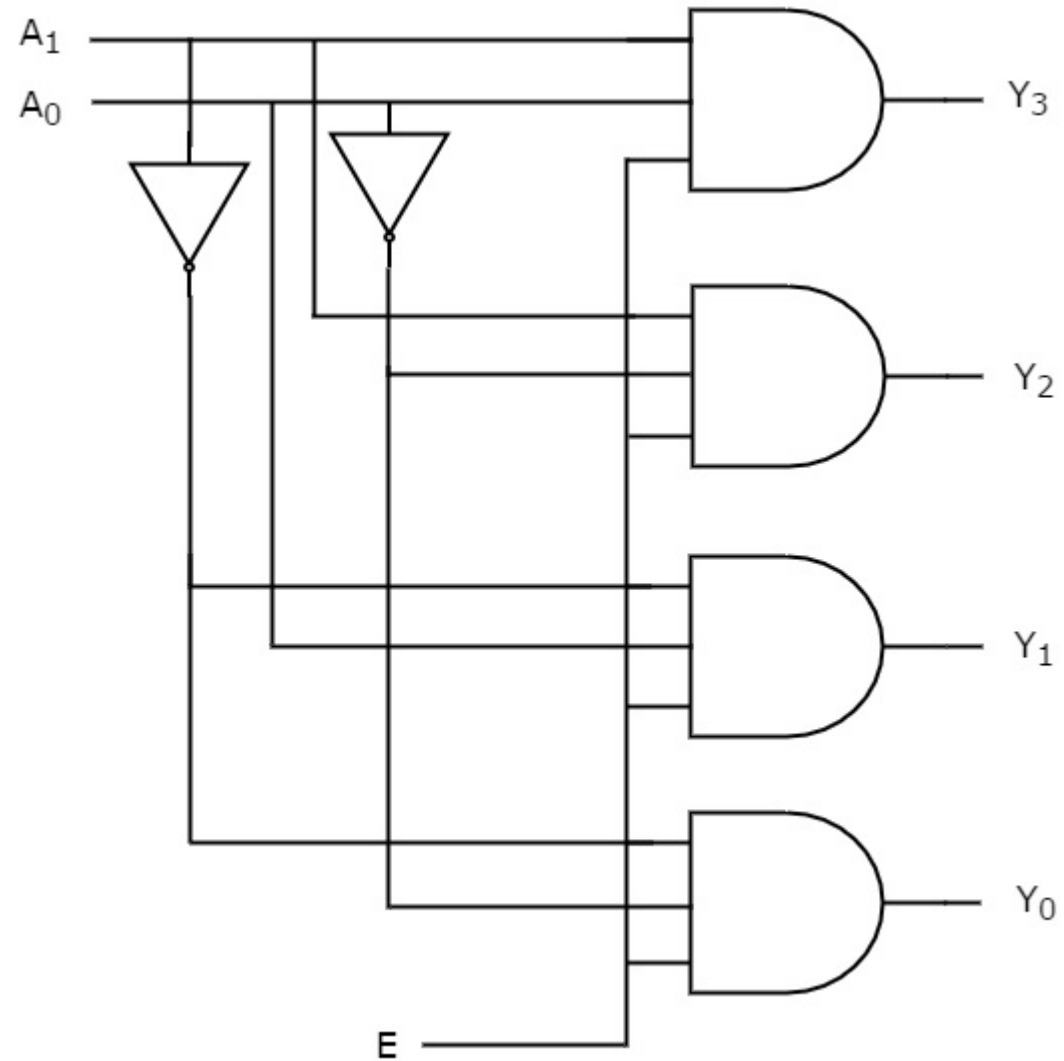
Decoder

$$Y_0 = E \overline{A_1} \overline{A_0}$$

$$Y_1 = E \overline{A_1} A_0$$

$$Y_2 = E A_1 \overline{A_0}$$

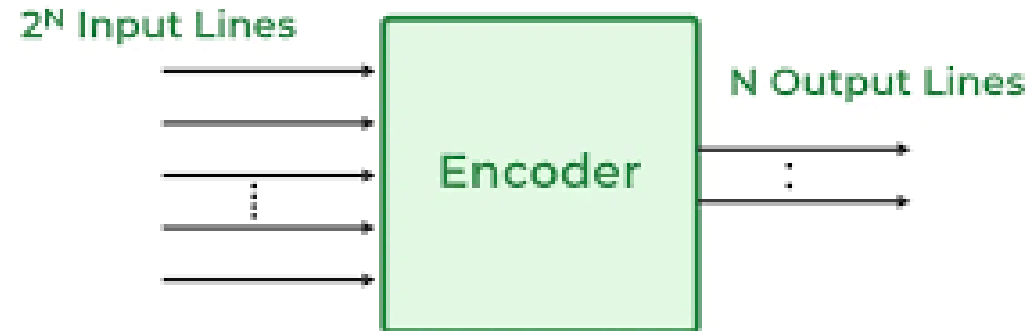
$$Y_3 = E A_1 A_0$$



Encoder

An Encoder performs the reverse operation of a decoder.

- It converts 2^n input lines (where only one input is active at a time) into an n -bit binary code output.
- Example: A 4-to-2 encoder takes 4 inputs and generates a 2-bit binary output.



Encoder



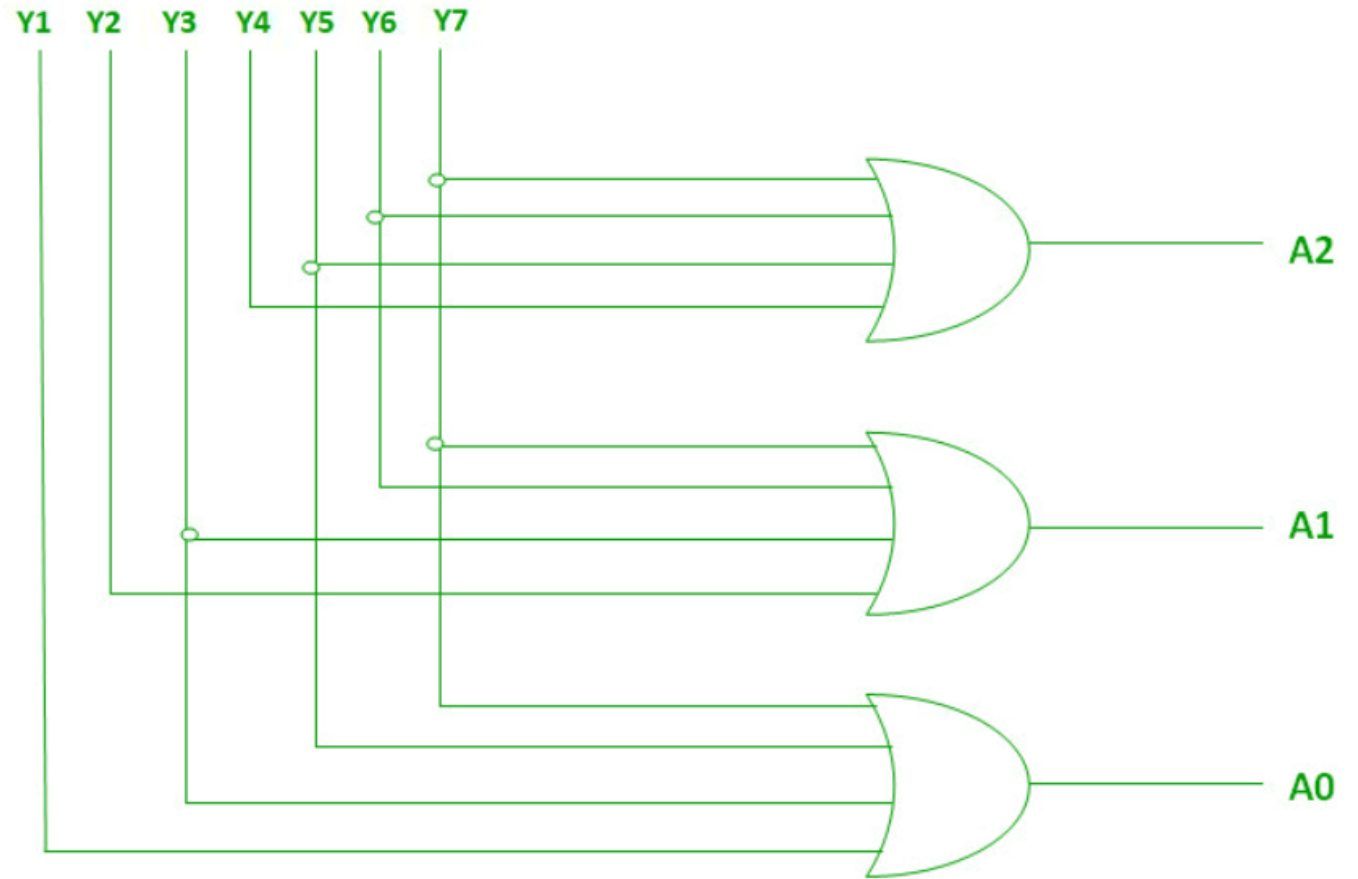
INPUTS								OUTPUTS		
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Encoder

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

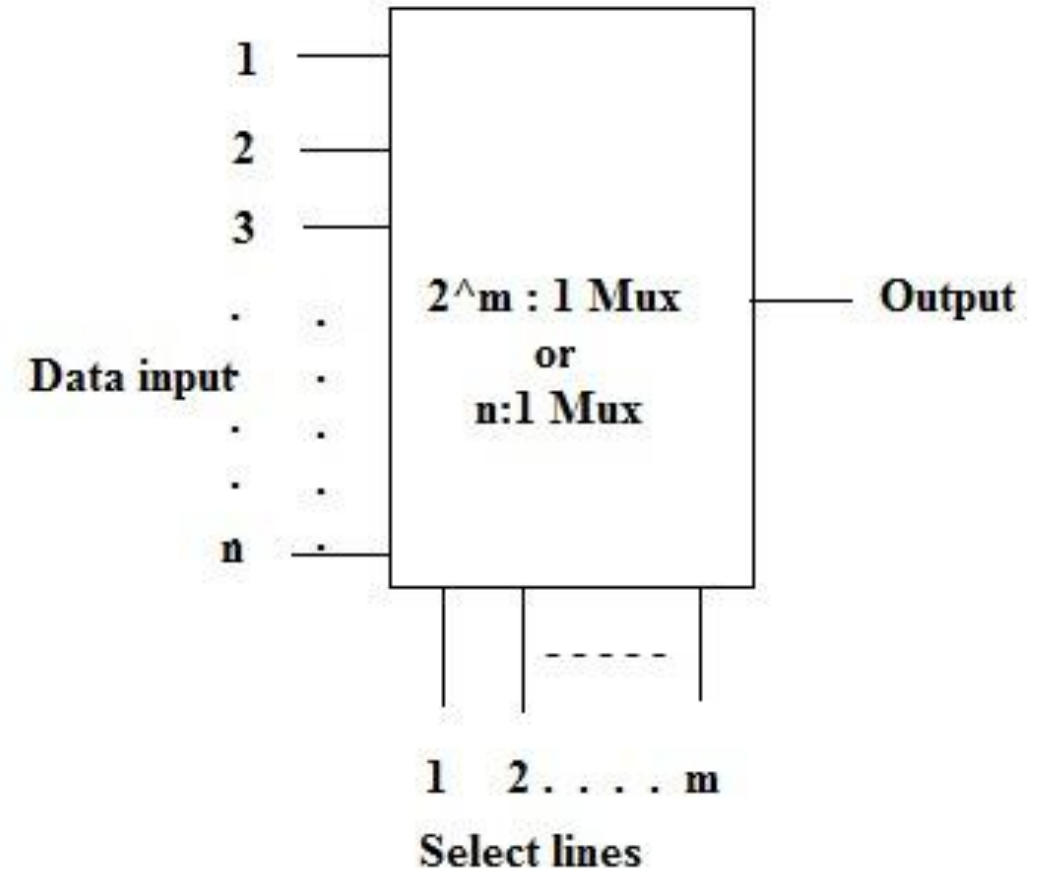
$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$



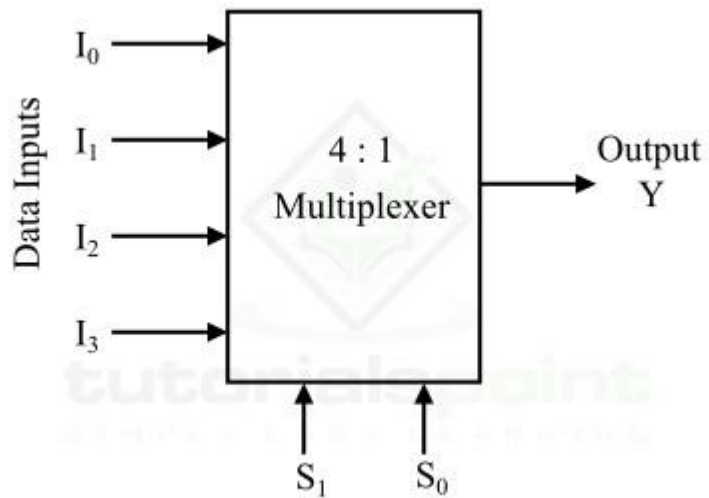
Multiplexer

A Multiplexer (MUX) selects one input from multiple input signals. The selected input is passed to the single output. The choice of input is controlled by selection lines.



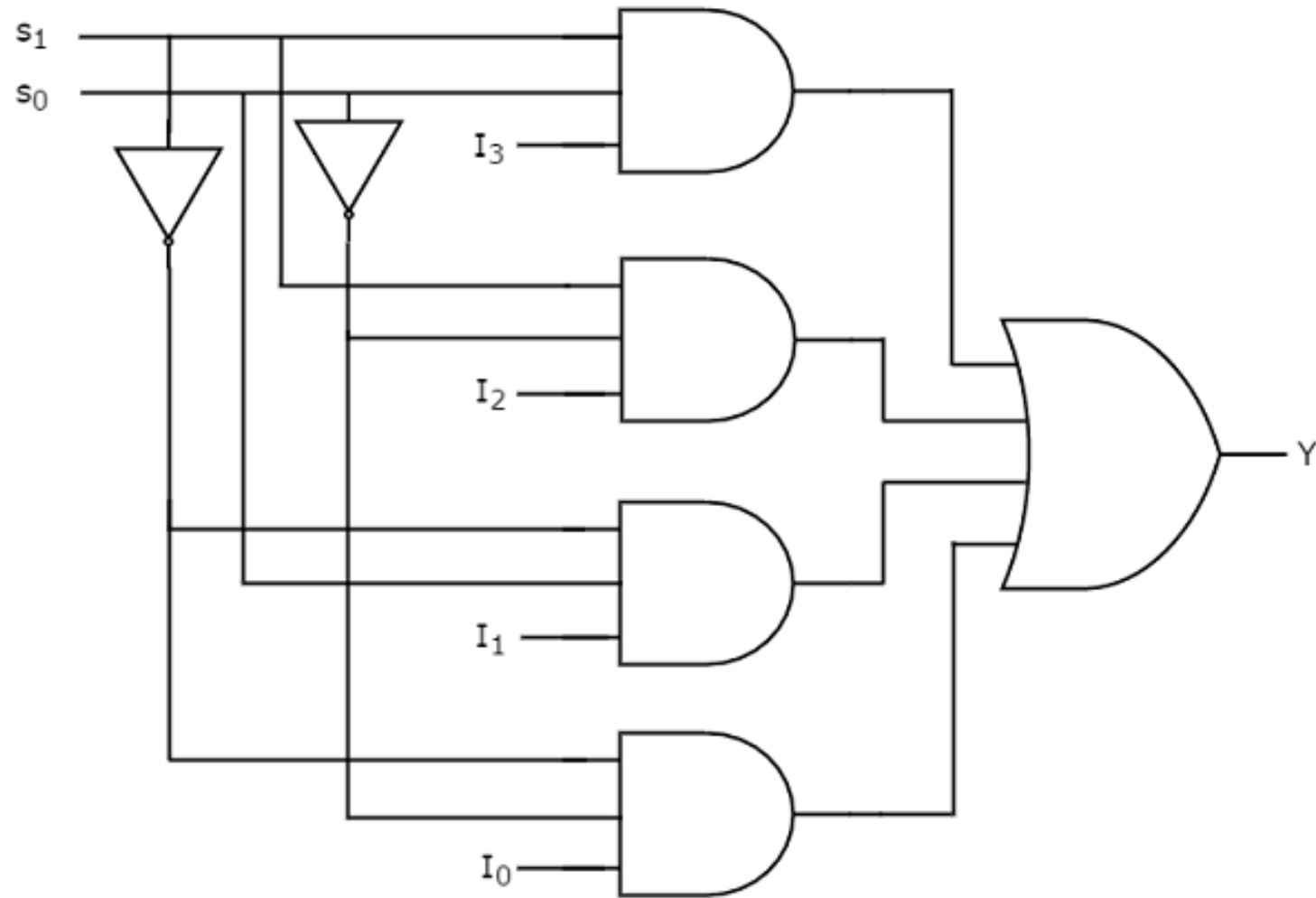
Multiplexer

Example: A 4-to-1 MUX has 4 inputs, 2 select lines, and 1 output.



Selection Lines		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Multiplexer



Comparator

A Comparator is a combinational logic circuit. It compares two binary numbers bit by bit. It produces outputs indicating whether:

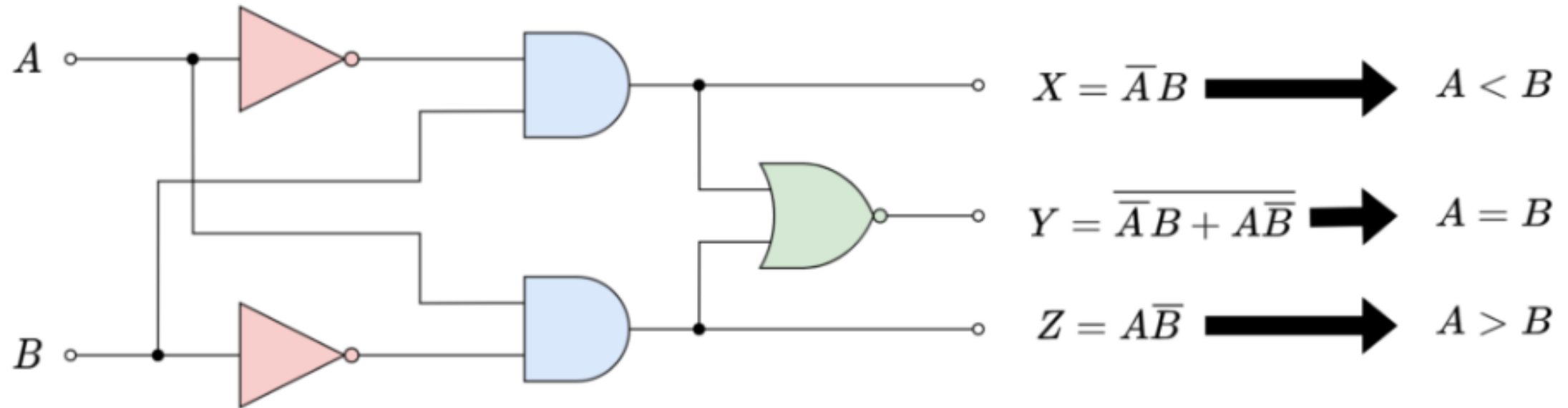
- They are equal,
- The first one is greater,
- The first one is less.



Comparator

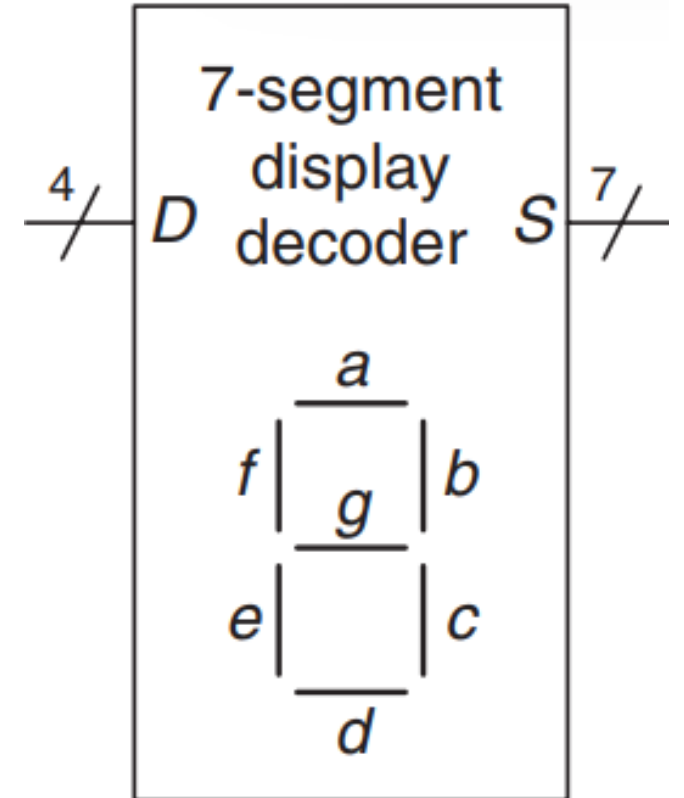
Inputs		Outputs		
B	A	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

Comparator

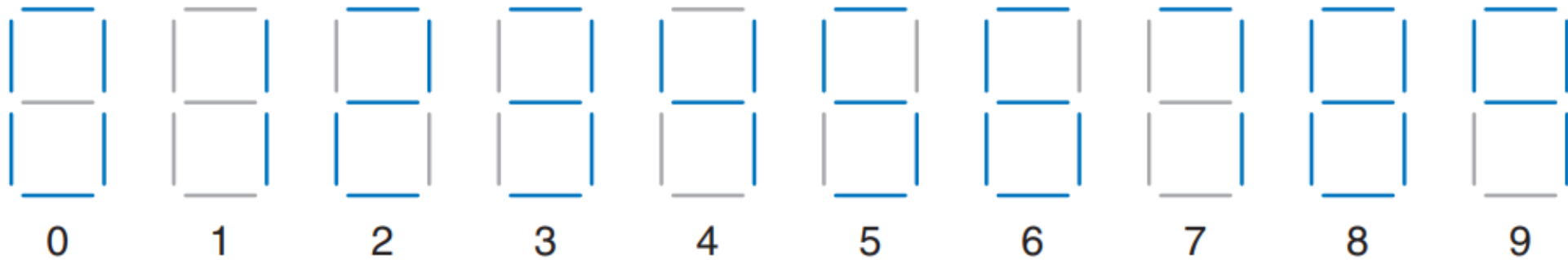


Seven-Segment Display Decoder

A seven-segment display decoder takes a 4-bit data input D3:0 and produces seven outputs to control light-emitting diodes to display a digit from 0 to 9. The seven outputs are often called segments a through g, or Sa–Sg.

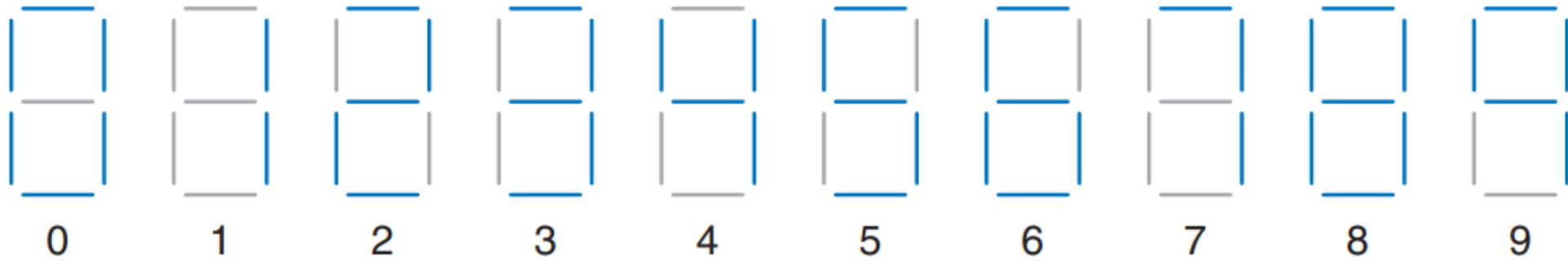


Seven-Segment Display Decoder



$D_{3:0}$	S_a	S_b	S_c	S_d	S_e	S_f	S_g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1

Seven-Segment Display Decoder



$D_{3:0}$	S_a	S_b	S_c	S_d	S_e	S_f	S_g
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	0	0	1	1
others	0	0	0	0	0	0	0

Seven-Segment Display Decoder

S_a $D_{3:2}$	00	01	11	10
$D_{1:0}$ 00	1	0	X	1
01	0	1	X	1
11	1	1	X	X
10	1	1	X	X

$$S_a = D_1 + D_3 + D_2 D_0 + \overline{D_2} \overline{D_0}$$